

ARAmame Arts - Full Repository

Architecture, Functioning & Request Processing Flow

Repository: <https://github.com/ArtecCo/pantings>

Branch reviewed: `main`

Review type: Source-level architecture and flow review

Important: This document is reconstructed from the repository source. I did not run the application or execute live API requests during this review.

1. Executive Summary

ARAmame Arts is a web-based painting marketplace/application composed of three major parts:

1. Customer frontend

- React
- Vite
- React Router
- Runs the public painting marketplace, authentication, cart, order submission, order tracking and account pages.

2. Admin frontend

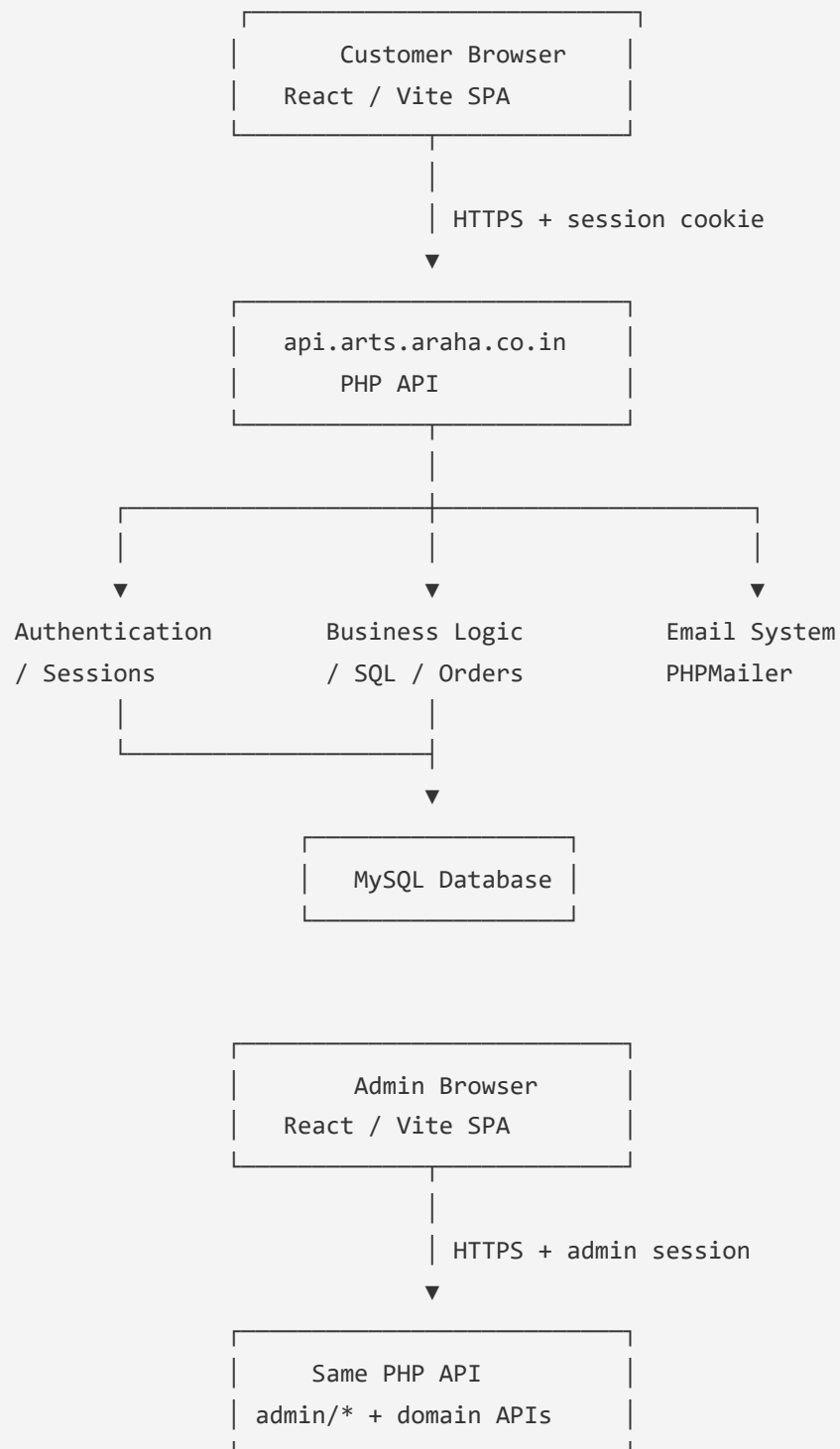
- Separate React/Vite application
- Provides dashboard, painting management, order management, metadata, FAQs, administrator management and maintenance mode.

3. PHP backend API

- Direct PHP endpoint architecture
- MySQL database through PDO

- Cookie/session-based authentication
- Separate customer and administrator sessions
- Email notifications through PHPMailer
- Application-level IP rate limiting
- SQL migrations for evolving the database schema

At a high level:



The repository root is split into `admin`, `api`, and `customer`, with several root-level manual test HTML files.

2. Repository Structure

The repository is essentially a monorepo containing two frontend applications and one backend.

```
pantings/
|
├─ admin/
|   ├── .env.example
|   ├── .gitignore
|   ├── README.md
|   ├── eslint.config.js
|   ├── index.html
|   ├── package.json
|   ├── package-lock.json
|   ├── public/
|   |   ├── .htaccess
|   |   ├── aa-icon.svg
|   |   ├── aa-og-img.svg
|   |   └─ aa-ogimage.png
|   └─ src/
|       ├── App.jsx
|       ├── App.css
|       ├── AdminBrandFixes.css
|       ├── animations.css
|       ├── index.css
|       ├── maintenance-button.css
|       ├── metadata-enhancer.js
|       ├── assets/
|       ├── components/
|       |   ├── PaintingSizeOptions.jsx
|       |   ├── Toast.jsx
|       |   ├── UniversalDropdown.jsx
|       |   ├── UniversalDropdown.css
|       |   └─ UniversalPrompt.jsx
```

```
| | | └─ UniversalPrompt.css
| | |
| | └─ config/
| |   └─ api.js
| |
| └─ pages/
|   ├── AddPainting.jsx
|   ├── AdminLogin.jsx
|   ├── AdminRegister.jsx
|   ├── Administrators.jsx
|   ├── EditPainting.jsx
|   ├── FAQs.jsx
|   ├── FAQs.css
|   ├── Metadata.jsx
|   ├── OrderDetails.jsx
|   ├── OrderDetails.css
|   ├── OrderHeritageFixes.css
|   ├── OrderStatusManager.jsx
|   ├── OrderStatusManager.css
|   ├── Orders.jsx
|   ├── Orders.css
|   ├── PaintingList.jsx
|   ├── ResetPassword.jsx
|   ├── Settings.jsx
|   └─ ...
|
└─ api/
  ├── .env.example
  ├── MAIL_SETUP.md
  ├── index.php
  |
  ├── admin/
  |   ├── _common.php
  |   ├── create-invitation.php
  |   ├── dashboard.php
  |   ├── logout.php
  |   ├── me.php
  |   ├── register.php
  |   ├── request-otp.php
  |   ├── request-password-reset.php
  |   └─ verify-otp.php
  |
  ├── auth/
  |   ├── _common.php
  |   ├── admin-session.php
  |   ├── login.php
  |   └─ logout.php
```

```
| | └─ me.php
| | └─ register.php
| | └─ require-admin.php
| | └─ reset-password.php
| | └─ session.php
| | └─ user-login.php
| | └─ user-logout.php
| | └─ user-register.php
| | └─ user-session.php
| | └─ verify-otp.php
|
| └─ cart/
|   └─ add-item.php
|   └─ get.php
|   └─ remove-item.php
|   └─ update-item.php
|
| └─ categories/
|   └─ create.php
|   └─ list.php
|
| └─ config/
|   └─ database.php
|   └─ google.php
|
| └─ faqs/
|   └─ admin.php
|   └─ list.php
|
| └─ frames/
|   └─ create.php
|   └─ list.php
|
| └─ lib/
|   └─ PHPMailer/
|
| └─ mail/
|   └─ Mailer.php
|   └─ Templates.php
|   └─ groups.php
|   └─ index.php
|
| └─ metadata/
|   └─ delete.php
|   └─ update.php
|
| └─ migrations/
```

```
| | | └─ 001_google_accounts.sql
| | | └─ 002_order_pricing.sql
| | | └─ 003_order_discount.sql
| | | └─ 004_painting_size_options.sql
| | | └─ 005_cart_item_size_option.sql
| | | └─ 006_order_contact_and_size.sql
| | | └─ 007_dedupe_painting_size_options.sql
| | | └─ 008_maintenance_mode.sql
| | | └─ 009_mail_groups.sql
| | | └─ 010_password_resets.sql
| | |
| | └─ orders/
| | | └─ cancel.php
| | | └─ create.php
| | | └─ events.php
| | | └─ get.php
| | | └─ history.php
| | | └─ list.php
| | | └─ my-orders.php
| | | └─ release-price.php
| | | └─ update-status.php
| | |
| | └─ paintings/
| | | └─ create.php
| | | └─ customer-get.php
| | | └─ delete-image.php
| | | └─ get.php
| | | └─ images-list.php
| | | └─ list.php
| | | └─ set-primary-image.php
| | | └─ update.php
| | | └─ upload-images.php
| | |
| | └─ security/
| | | └─ rate-limit.php
| | |
| | └─ sizes/
| | | └─ create.php
| | | └─ list.php
| | |
| | └─ system/
| | | └─ maintenance-status.php
| | | └─ maintenance-toggle.php
| | | └─ maintenance.php
| |
| └─ customer/
| | └─ .env.example
```

```
| |├─ .gitignore
| |├─ README.md
| |├─ eslint.config.js
| |├─ index.html
| |├─ package.json
| |├─ package-lock.json
| |├─ public/
| | |├─ .htaccess
| | |├─ aa-icon.svg
| | |├─ aa-og-img.svg
| | |├─ aa-ogimage.png
| | |├─ art-viewer-touch.js
| | |├─ index.php
| | |├─ rate-limit-toast.js
| | |├─ robots.txt
| | |└─ sitemap.xml
| |└─ src/
| |   ├─ App.jsx
| |   ├─ App.css
| |   ├─ animations.css
| |   ├─ cart-count.js
| |   ├─ cart-redesign.css
| |   ├─ collection-redesign.css
| |   ├─ fixes.css
| |   ├─ index.css
| |   ├─ main.jsx
| |   ├─ painting-details-redesign.css
| |   ├─ responsive.css
| |   |
| |   ├─ assets/
| |   |
| |   ├─ components/
| |   |   ├─ Layout.jsx
| |   |   ├─ Toast.jsx
| |   |   ├─ ToastProvider.jsx
| |   |   ├─ UniversalDropdown.jsx
| |   |   ├─ UniversalDropdown.css
| |   |   ├─ UniversalPrompt.jsx
| |   |   └─ UniversalPrompt.css
| |   |
| |   ├─ config/
| |   |   └─ api.js
| |   |
| |   └─ pages/
| |       ├─ Account.jsx
| |       └─ Account.css
```

```
├── Cart.jsx
├── FAQ.jsx
├── FAQ.css
├── Home.jsx
├── Login.jsx
├── Login.css
├── OrderDetails.jsx
├── OrderDetails.css
├── PaintingDetails.jsx
├── Paintings.jsx
├── PrivacyPolicy.jsx
├── ResetPassword.jsx
├── TermsAndConditions.jsx
├── TrackOrders.jsx
├── TrackOrders.css
├── README.md
├── test-admin.html
├── test-admin-register.html
└── test-register.html
```

- React
- React DOM
- React Router DOM
- Axios
- Vite
- ESLint

The package currently declares React 19.2.8 , React DOM 19.2.8 , React Router DOM 7.18.3 , and Vite 8.3.0 .

The application code inspected for the main commerce flows primarily uses the browser's native `fetch()` API.

Admin

The admin application is a separate React/Vite project under `admin/` .

The two frontend applications are deliberately separated rather than being one React application with an admin namespace. The admin has its own route tree and its own session model.

Backend

The backend is:

```
PHP
MySQL
PDO
PHP sessions
PHPMailer
JSON HTTP responses
```

The database connection is created with PDO using a MySQL DSN and prepared statements with emulated prepares disabled.

External services / browser integrations

The customer login integrates Google Identity Services for Google Sign-In. The browser obtains a Google credential and posts it to the backend `auth/user-login.php` endpoint.

4. Architectural Style

This is **not** an MVC framework such as Laravel, Symfony, CodeIgniter, etc.

The backend uses a lightweight endpoint-per-file architecture.

For example:

```
GET /api/paintings/list.php
GET /api/paintings/get.php?id=123

POST /api/cart/add-item.php
POST /api/cart/update-item.php
POST /api/cart/remove-item.php

POST /api/orders/create.php
POST /api/orders/cancel.php

POST /api/orders/update-status.php

POST /api/paintings/create.php
POST /api/paintings/update.php
POST /api/paintings/upload-images.php
```

Each PHP file is effectively a route/controller.

There is no large centralized router deciding which application service to invoke.

The pattern is closer to:

```
HTTP request
  ↓
specific PHP endpoint
  ↓
shared common/auth/config includes
  ↓
validation
  ↓
SQL
  ↓
side effects
  ↓
JSON response
```

This makes the system relatively easy to trace feature-by-feature because the endpoint name usually corresponds directly to the UI operation.

5. Customer Frontend Architecture

Entry point

The customer application begins in:

```
customer/src/main.jsx
```

and the main application routing and global behavior are in:

```
customer/src/App.jsx
```

The app defines the main customer routes and wraps them in global layout/toast functionality.

6. Customer Routes

The current customer route map is:

```
/
  Home

/paintings
  Paintings collection

/paintings/:id
  Individual painting

/faq
  FAQ

/login
  Login / registration / password reset UI

/reset-password
```

Password reset page

/account

Customer account

/cart

Cart + checkout/order request

/track-orders

Customer order list/tracking

/track-orders/:id

Individual order details

/privacy-policy

Privacy policy

/terms-and-conditions

Terms and conditions

These routes are explicitly defined in `customer/src/App.jsx`.

7. Global Customer Application Behavior

`App.jsx` does more than route pages.

It contains three important global systems.

7.1 Metadata manager

The application dynamically manages:

```
document.title
meta description
Open Graph metadata
Twitter card metadata
canonical URL
```

For a painting detail route, it additionally fetches:

```
paintings/get.php?id=<painting-id>
```

and derives the painting's name/description for page metadata.

This means a painting detail page can trigger an API request specifically for SEO metadata in addition to the painting page's own data loading.

8. Customer Scroll Management

The collection page uses session storage to preserve scroll position when the user navigates away and returns.

Conceptually:

```
/paintings
  ↓
user scrolls
  ↓
navigate to painting
  ↓
scroll position stored in sessionStorage
  ↓
Back navigation
  ↓
/paintings restored to previous position
```

This is implemented by `ScrollManager` in `App.jsx`.

9. Maintenance Mode

The public customer application continuously checks:

```
system/maintenance.php
```

The code polls the endpoint approximately every two seconds.

If the backend reports:

```
maintenance_mode = true
```

the customer application replaces the normal UI with a full-screen maintenance interface.

There is also a manual:

```
Check again
```

button in the maintenance state.

The backend also has explicit maintenance endpoints:

```
api/system/maintenance.php  
api/system/maintenance-status.php  
api/system/maintenance-toggle.php
```

10. Customer API Configuration

The customer frontend uses:

```
customer/src/config/api.js
```

Its behavior is:

```
localhost / 127.0.0.1  
  ↓  
VITE_API_BASE_URL  
  ↓  
default:  
http://localhost/paintings/api  
  
production host
```



```
https://api.arts.araha.co.in
```

The API helper then converts:

```
paintings/list.php
```

into:

```
https://api.arts.araha.co.in/paintings/list.php
```

in production.

This is important because the production website and API are separate origins.

11. Customer Authentication Architecture

The customer authentication backend uses a PHP session.

The common authentication file is:

```
api/auth/_common.php
```

It:

- starts a PHP session
- defines the session cookie
- uses `HttpOnly`
- supports `Secure`
- supports configurable `SameSite`
- configures CORS
- allows credentials
- parses JSON requests
- provides common JSON response functionality
- provides `requireCustomer()`

The customer session is named:

```
painting_marketplace_session
```

and authenticated users are identified through:

```
$_SESSION['user_id']  
$_SESSION['user_type'] === 'customer'
```

This is **cookie/session authentication**, not a JWT/bearer-token architecture.

12. Customer Login Flow

The frontend login page supports:

```
Email + password  
Google Sign-In  
Registration  
Forgot password
```

Email login sends:

```
POST /auth/user-login.php
```

with approximately:

```
{  
  "type": "email",  
  "email": "customer@example.com",  
  "password": "..."  
}
```

Google login sends:

```
POST /auth/user-login.php
```

with:


```
{  
  "type": "google",  
  "token": "<google credential>"  
}
```

The browser uses:

```
credentials: include
```

so the session cookie set by the API can be used on subsequent API requests.

13. Login Processing Flow

```
Customer enters credentials  
↓  
Login.jsx  
↓  
POST /auth/user-login.php  
↓  
PHP session/auth common layer  
↓  
authentication + database work  
↓  
session established  
↓  
JSON success response  
↓  
customer browser redirects to /account
```

The frontend does not store an authorization token in localStorage as its main authentication mechanism.

The backend session is the source of authentication state.

14. Customer Painting Collection Flow

The collection is primarily backed by:

```
GET /paintings/list.php
```

The repository contains separate painting API endpoints for:

```
list
get one painting
customer-specific get
create
update
upload images
list images
set primary image
delete image
```

A simplified flow:

```
Customer visits /paintings
↓
Paintings.jsx
↓
GET /paintings/list.php
↓
PHP API
↓
MySQL
↓
JSON list of paintings
↓
React renders collection
```

15. Painting Detail Flow

When a user visits:

```
/paintings/:id
```

the `PaintingDetails.jsx` page calls:

```
GET /paintings/get.php?id=<id>
```

The returned painting object contains information such as:

```
name
description
artist
category
price
discount price
dimensions
medium
frame
gold details
images
size options
```

The component then renders:

```
painting title
artist
price
dimensions
gallery
thumbnail gallery
image viewer
zoom
pan
```

The detail page's source explicitly loads `paintings/get.php` and builds its gallery from `painting.images` or `image_url` .

16. Painting Viewer

The painting detail page also includes a full-screen artwork viewer.

It supports:

```
Open viewer
Close viewer
Previous image
Next image
Zoom in
Zoom out
Reset zoom
Mouse/pointer dragging while zoomed
Keyboard navigation
Escape to close
Mouse wheel zoom
```

The image is transformed rather than replaced by another page.

The body scroll is disabled while the viewer is open.

17. Add-to-Cart Flow

From a painting detail page:

```
User clicks Add to Cart
    ↓
PaintingDetails.jsx
    ↓
POST /cart/add-item.php
    ↓
JSON:
{
  painting_id: <id>,
  quantity: 1
}
    ↓
credentials: include
    ↓
backend checks session
    ↓
cart item inserted/updated
```

```
↓
success JSON
↓
toast
↓
navigate to /cart
```

If the API returns HTTP `401` , the user is redirected to:

```
/login
```

The actual call is visible in `PaintingDetails.jsx` .

18. Cart Architecture

The cart backend is divided into:

```
cart/add-item.php
cart/get.php
cart/update-item.php
cart/remove-item.php
```

The customer cart page orchestrates these endpoints.

The cart supports painting-specific size options.

Conceptually:

```
Painting
|
├─ Standard size
├─ Small
├─ Medium
└─ Large
```

The selected size option determines the cart item's price.

19. Cart Load Flow

When `/cart` loads:

```
Cart.jsx
  ↓
GET /cart/get.php
  ↓
credentials: include
  ↓
customer session validation
  ↓
cart lookup
  ↓
painting lookup
  ↓
size options loaded
  ↓
JSON response
  ↓
React renders cart
```

If the response is `401` :

```
navigate('/login')
```

The cart page implements this behavior directly.

20. Cart Size Selection Flow

For each item:

```
cart item
  ↓
available painting size options
  ↓
UniversalDropdown
  ↓
```

```
user chooses size
↓
POST /cart/update-item.php
```

The request includes:

```
{
  "cart_item_id": 123,
  "quantity": 1,
  "size_option_id": 456
}
```

The cart reloads after the update.

21. Cart Removal Flow

```
User clicks Remove
↓
confirmation prompt
↓
confirm
↓
POST /cart/remove-item.php
↓
cart item deleted
↓
cart reloaded
↓
cart count update event
↓
success toast
```

The cart uses a reusable `UniversalPrompt` for confirmation.

22. Order Submission Architecture

This application does **not** treat the initial checkout as an immediate payment transaction.

The cart page clearly treats this as:

Order Request

rather than:

Pay Now

The order begins in:

PENDING_ACCEPTANCE

and is reviewed after submission.

23. Exact Order Creation Flow

This is one of the most important backend processes in the repository.

The customer submits:

POST /orders/create.php

with:

```
{
  "shipping_name": "...",
  "shipping_phone": "...",
  "shipping_email": "...",
  "shipping_address": "...",
  "shipping_city": "...",
  "shipping_state": "...",
  "shipping_postal_code": "...",
  "shipping_country": "..."
}
```


The customer cart does exactly this after the user submits the order form.

24. Server-Side Order Creation Pipeline

The backend flow is:

```
POST /orders/create.php
↓
require customer
↓
read JSON request
↓
validate shipping/contact fields
↓
status = PENDING_ACCEPTANCE
↓
BEGIN TRANSACTION
↓
lock current cart rows FOR UPDATE
↓
load paintings
↓
validate paintings are active
↓
validate quantities
↓
resolve selected painting size
↓
resolve standard size if none selected
↓
calculate unit price
↓
calculate base amount
↓
generate order number
↓
INSERT orders
↓
INSERT order_items
↓
INSERT order_status_history
↓
```

```
DELETE cart items
  ↓
COMMIT
  ↓
send email notifications
  ↓
return JSON 201
```

This exact structure is implemented in `api/orders/create.php` .

25. Why **FOR UPDATE** Matters in Order Creation

The order creation query uses:

```
FOR UPDATE
```

while reading cart rows.

This means the transaction locks the affected cart rows while the order is being built.

Conceptually:

```
Cart read
  ↓
row lock
  ↓
price/availability calculation
  ↓
order insert
  ↓
cart deletion
  ↓
commit
```

This is designed to reduce race conditions around concurrent order submission.

26. Order Pricing Model

The backend currently resolves the price roughly in this order:

```
Selected size option exists?  
  |  
  └─ YES → use size option price  
  |  
  └─ NO  
      ↓  
    standard size option  
      ↓  
    if still unavailable:  
      ↓  
    discount_price if valid  
      ↓  
    otherwise base price
```

The order then stores its own pricing values.

The SQL insert includes fields such as:

```
subtotal  
base_amount  
customization_amount  
shipping_amount  
delivery_amount  
discount_amount  
total_amount
```

At the current order creation path:

```
customization_amount = 0  
deliveryAmount = 0  
discountAmount = 0  
  
totalAmount = baseAmount
```

The exact implementation is in `api/orders/create.php`.

27. Why Order Items Store Size Information

When an order item is created, the backend stores not merely:

```
painting_id
```

but also snapshot-like size information:

```
size_option_id
size_name
size_width
size_height
size_unit
painting_name
quantity
unit_price
total_price
```

This means historical orders retain the relevant pricing/size information even if a painting's current configuration changes later.

The database evolution for this feature is reflected in the migration sequence:

```
004_painting_size_options.sql
005_cart_item_size_option.sql
006_order_contact_and_size.sql
007_dedupe_painting_size_options.sql
```

28. Order Number Generation

The backend creates an order number in the form:

```
ORD-<random hexadecimal value>
```

It uses random bytes rather than relying only on the database numeric ID.

So there are effectively two identifiers:

```
database order ID
order number shown/communicated to users
```

29. Order Status History

Every important status change is recorded separately in:

```
order_status_history
```

Initial order creation creates:

```
old_status = NULL
new_status = PENDING_ACCEPTANCE
changed_by = customer user ID
changed_by_type = USER
notes = "Initial order request"
```

This creates an audit-style timeline for the order.

30. Cart Cleanup After Order Creation

After successfully creating:

```
orders
order_items
order_status_history
```

the backend deletes the user's cart items and commits the transaction.

Therefore:

```
Cart
↓
```

```
Order
↓
Cart emptied
```

The cart is not left as the permanent copy of the order.

The order tables become the authoritative historical representation.

31. Order Creation and Email Notifications

Only after the transaction commits does the backend call:

```
sendNewOrderNotifications(...)
```

This is an important sequencing decision.

Conceptually:

```
Database transaction
↓
COMMIT SUCCESSFULLY
↓
send email notification
```

rather than:

```
send email
↓
maybe commit
```

The mail infrastructure is separated into:

```
api/mail/Mailer.php
api/mail/Templates.php
api/mail/groups.php
api/mail/index.php
```

with PHPMailer included under:

32. Customer Order Tracking

The customer order subsystem has:

```
orders/my-orders.php
orders/get.php
orders/events.php
orders/history.php
```

and the frontend contains:

```
TrackOrders.jsx
OrderDetails.jsx
```

The conceptual flow is:

```
/track-orders
  ↓
GET my-orders.php
  ↓
user-specific order list
  ↓
click order
  ↓
/track-orders/:id
  ↓
GET order details
  ↓
order information + status history/events
```

The complete order API surface is visible in `api/orders` .

33. Customer Cancellation

The customer cancellation request is:

```
POST /orders/cancel.php
```

The backend:

1. Requires customer authentication.
 2. Looks up the order belonging to that customer.
 3. Locks the order with `FOR UPDATE` .
 4. Checks current status.
 5. Changes the status to `CANCELLED` .
 6. Inserts status history.
 7. Commits.
 8. Sends a status notification.
-

34. Current Cancellation Rule

The current backend blocks cancellation when status is:

```
CANCELLED  
PAID  
PROCESSING  
DISPATCHED  
DELIVERED
```

Therefore, according to the current implementation:

```
PENDING_ACCEPTANCE → cancellable  
ACCEPTED             → cancellable  
PAYMENT_DUE          → cancellable  
REJECTED             → cancellable  
PAID                 → not cancellable  
PROCESSING           → not cancellable  
DISPATCHED          → not cancellable
```


DELIVERED	→ not cancellable
CANCELLED	→ not cancellable

This is an important implementation detail.

In particular, **REJECTED** **is currently not blocked by** `cancel.php`. If the intended business rule is that once an artist rejects an order the customer must never be able to cancel it, the current backend does not yet enforce that rule.

35. Admin Application Architecture

The admin app is completely separate from the customer app.

Its routing is implemented in:

```
admin/src/App.jsx
```

The major routes include:

```
/login
/reset-password
/register

/
/orders
/orders/:id

/paintings
/paintings/new
/paintings/:id/edit

/metadata
/settings
/faqs
/administrators
```

There are also currently exposed placeholder sections for:

```
/customers  
/payments  
/reports  
/audit
```

The admin application itself explicitly describes these sections as areas that will be connected to the marketplace API next.

36. Admin Route Protection

Admin pages use a React-level `ProtectedRoute` .

The sequence is:

```
Admin opens protected URL  
    ↓  
ProtectedRoute  
    ↓  
GET /auth/admin-session.php  
    ↓  
backend checks admin session  
    ↓  
two_factor_verified?  
    ↓  
YES → render page  
NO  → redirect to /login
```

The frontend explicitly requires:

```
success  
authenticated  
two_factor_verified
```

before allowing the page to render.

37. Admin Backend Authentication

The backend also independently protects administrative API operations.

`api/admin/_common.php` establishes a different PHP session:

```
painting_marketplace_admin_session
```

It then requires:

```
admin_user_id exists
admin_user_type === admin
admin_2fa_verified === true
```

The `requireAdmin()` function rejects requests that do not satisfy those conditions.

This means the application has two layers of admin protection:

```
React route guard
+
backend authorization
```

The backend is ultimately authoritative.

38. Admin Session vs Customer Session

There are deliberately different cookies:

```
Customer:
painting_marketplace_session

Admin:
painting_marketplace_admin_session
```

This separation prevents the two authentication contexts from being represented by the same session namespace.

39. Admin Dashboard

The admin dashboard requests:

```
GET /admin/dashboard.php
```

and displays figures such as:

```
pending orders
paintings
customers
revenue
```

The dashboard page also contains the maintenance mode control.

The admin app's current dashboard source implements this request directly.

40. Admin Maintenance Mode

The admin interface can call:

```
GET /system/maintenance.php
POST /system/maintenance-toggle.php
```

The flow is:

```
Admin dashboard
  ↓
Maintenance Mode control
  ↓
confirmation prompt
  ↓
POST maintenance-toggle.php
  ↓
backend validates admin
```

```
↓
maintenance state changes
↓
customer frontend discovers state
↓
customer site enters/exits maintenance mode
```

41. Admin Painting Management

The admin painting subsystem includes:

```
PaintingList
AddPainting
EditPainting
PaintingSizeOptions
```

The backend has:

```
paintings/create.php
paintings/update.php
paintings/list.php
paintings/get.php
paintings/customer-get.php
paintings/upload-images.php
paintings/images-list.php
paintings/set-primary-image.php
paintings/delete-image.php
```

Conceptually:

```
Admin form
↓
POST JSON / multipart request
↓
painting endpoint
↓
admin auth
↓
validation
```

```
↓  
MySQL  
↓  
JSON response  
↓  
admin UI refresh
```

42. Painting Image Management

Painting images are treated as a distinct subsystem rather than simply a single image column.

The backend has explicit operations:

```
upload-images.php  
images-list.php  
set-primary-image.php  
delete-image.php
```

This implies a model where a painting can have multiple image records.

The customer UI also expects an array of images and provides a gallery/thumb viewer.

43. Size Management

Sizes are a first-class data feature.

The API provides:

```
sizes/create.php  
sizes/list.php
```

And painting-specific size relationships are represented in:

```
painting_size_options
```

The migration history shows that size support was progressively added to:

```
paintings  
cart_items  
orders  
order_items
```

44. Frame Management

The backend contains:

```
frames/create.php  
frames/list.php
```

This is likely supporting selectable/configurable painting frame information. The exact implementation is endpoint-based.

45. Category Management

The API currently includes:

```
categories/create.php  
categories/list.php
```

Categories are also surfaced as painting metadata in the customer UI.

46. FAQ Management

The backend contains:

```
faqs/list.php  
faqs/admin.php
```

while the admin frontend has:

```
FAQs.jsx
```

This gives the application a data-driven FAQ layer rather than requiring FAQ text to be permanently hardcoded in the customer page.

47. Metadata Management

The backend metadata subsystem is:

```
metadata/update.php  
metadata/delete.php
```

and the admin frontend contains:

```
Metadata.jsx  
metadata-enhancer.js
```

This indicates that metadata is intended to be administratively editable rather than completely static.

48. Database Configuration

The main database file is:

```
api/config/database.php
```

Production configuration can be supplied via:


```
api/config/production.php
```

which is intentionally not committed.

The connection parameters can also be provided through environment variables.

The production configuration is designed to keep database credentials out of normal tracked source.

49. Database Connection Flow

The backend essentially does:

```
PHP endpoint
  ↓
require common/auth
  ↓
require database.php
  ↓
rate limiter
  ↓
load production.php or environment variables
  ↓
construct PDO MySQL DSN
  ↓
new PDO(...)
  ↓
prepared statements
```

PDO options include:

```
ERRMODE_EXCEPTION
FETCH_ASSOC
EMULATE_PREPARES = false
```

50. Core Database Entities

Based on the SQL used by the current backend, the important business tables include concepts such as:

```
users
carts
cart_items

paintings
painting_size_options

orders
order_items
order_status_history

administrators / admin users

categories
frames
faqs

password reset records
Google account linkage

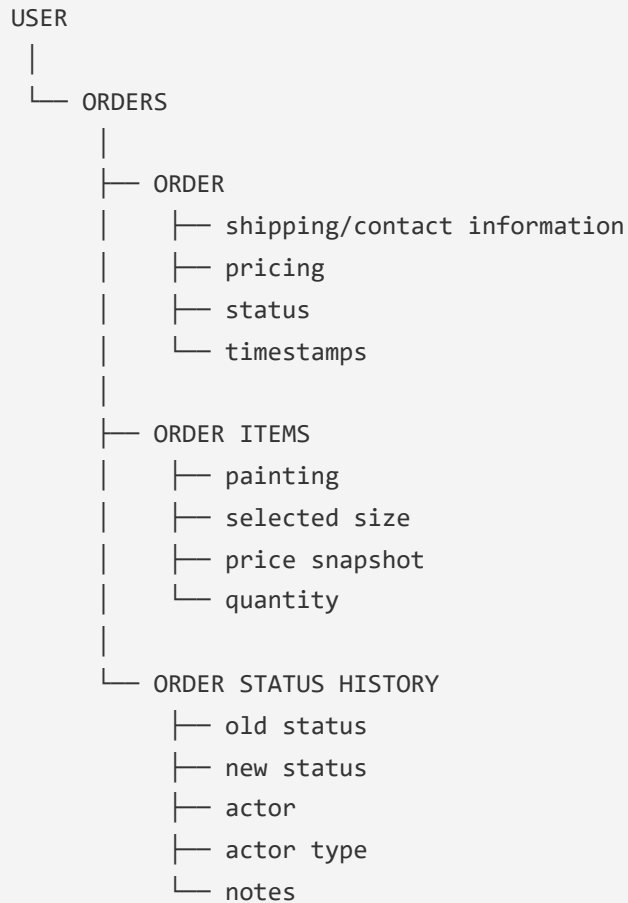
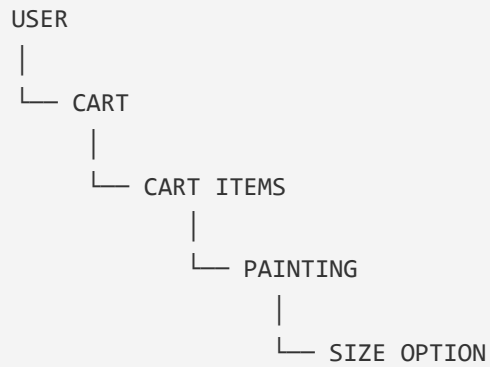
maintenance configuration
mail groups
metadata
```

The exact database schema has evolved through migration files rather than being represented by one single current SQL file.

The migration history contains ten visible migrations covering Google accounts, order pricing, discounts, sizes, order contact information, deduplication, maintenance mode, mail groups, and password resets.

51. Order Data Model

The basic conceptual relationship is:



This structure is directly reflected by the queries in `orders/create.php` and `orders/update-status.php` .

52. Order Lifecycle

The backend recognizes the following statuses:

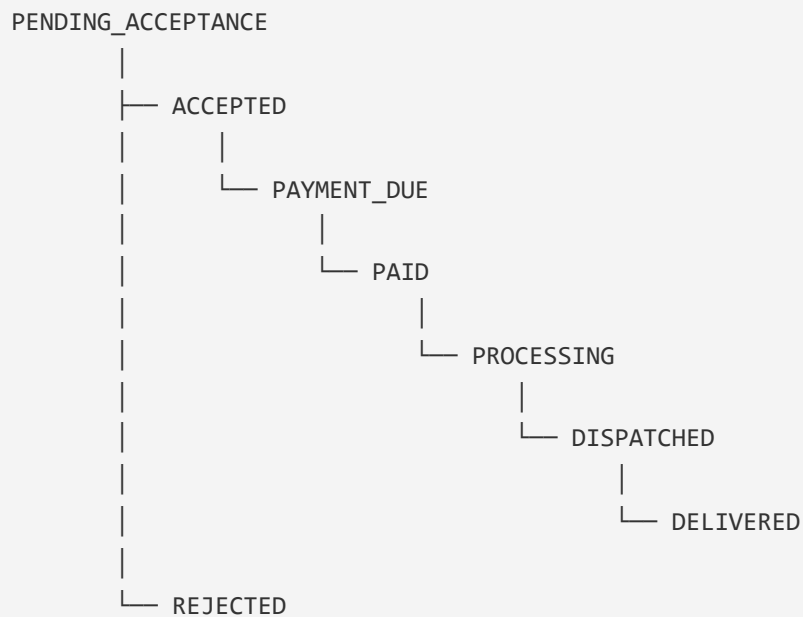
PENDING_ACCEPTANCE
ACCEPTED
PAYMENT_DUE
PAID
PROCESSING
DISPATCHED
DELIVERED
REJECTED

Customer cancellation introduces:

CANCELLED

The admin status endpoint explicitly validates the first set of statuses.

A conceptual lifecycle is therefore:



Customer may also transition certain states to:

CANCELLED

The actual transition rules are not fully encoded as a formal state machine; the status endpoint mainly validates that the requested status exists and blocks certain situations such as editing a cancelled order.

53. Admin Status Update Flow

The admin order status endpoint is:

```
POST /orders/update-status.php
```

The request contains approximately:

```
{
  "order_id": 123,
  "action": "ACCEPTED",
  "notes": "...
}
```

Processing:

```
Admin frontend
↓
POST update-status.php
↓
requireAdmin()
↓
validate order ID
↓
validate action
↓
lock order FOR UPDATE
↓
check previous status
↓
update status
↓
update artist_notes if supplied
↓
set relevant timestamp
↓
insert order_status_history
↓
COMMIT
↓
sendOrderStatusNotification()
```



JSON response

54. Important Admin Status Timestamps

When the status changes, the backend sets timestamps for certain milestones:

```
ACCEPTED    → accepted_at
PAID        → paid_at
DISPATCHED → dispatched_at
DELIVERED   → delivered_at
```

This allows the order to contain both:

```
current status
```

and:

```
historical milestone timestamps
```

55. Artist Notes

The status update endpoint accepts:

```
notes
```

and stores them in:

```
artist_notes
```

when supplied.

The same note is also inserted into the status history record.

This makes an artist/admin note visible both as the current order note and as part of the status-change history.

56. Email Architecture

The email system is a separate backend subsystem.

```
api/mail/  
├─ Mailer.php  
├─ Templates.php  
├─ groups.php  
└─ index.php
```

and:

```
api/lib/PHPMailer/
```

is present as the mail library.

At least two central application-level notification functions are referenced by order flows:

```
sendNewOrderNotifications()  
sendOrderStatusNotification()
```

57. System Maintenance Email Notification

Whenever the system is placed into maintenance mode, a system notification email is sent to the **System Updates** mail group.

57.1 Trigger

The notification is triggered from:

```
api/system/maintenance-toggle.php
```

After the maintenance mode state is successfully updated, the system sends the notification using the existing mail infrastructure.

57.2 Recipient Group

The notification is sent to the:

System Updates

mail group.

The group is managed through:

```
api/mail/groups.php
```

57.3 Email Template

The maintenance notification template is maintained in:

```
api/mail/Templates.php
```

The email informs the System Updates recipients that the system has been placed into maintenance mode.

57.4 Mail Flow

```
graph TD
    A[Admin enables Maintenance Mode] --> B[api/system/maintenance-toggle.php]
    B --> C[Maintenance state updated]
    C --> D[System Maintenance Email]
```



```
      v
System Updates Mail Group
      |
      v
api/mail/Mailer.php
      |
      v
PHPMailer / SMTP
```

58. New Order Email Flow

```
Customer submits order
      ↓
transaction succeeds
      ↓
COMMIT
      ↓
sendNewOrderNotifications()
      ↓
mail templates / mailer
      ↓
PHPMailer
      ↓
recipients
```

The key point is that email is invoked after the order transaction completes. The mail is sent to members of the mail group *ORDER_UPDATES*

59. Status Email Flow

When an admin changes an order's status:

```
Admin changes status
      ↓
DB update
```

```
↓  
history insert  
↓  
COMMIT  
↓  
sendOrderStatusNotification()
```

When a customer cancels an order, the same notification mechanism is used for the cancellation status.

60. CORS Architecture

Because the frontend and API are different origins in production, the backend explicitly implements CORS.

The customer auth common layer allows configured origins and includes:

```
https://arts.araha.co.in  
https://artsadmin.araha.co.in
```

along with localhost development origins.

Credentials are explicitly allowed:

```
Access-Control-Allow-Credentials: true
```

This is necessary because the application uses session cookies across the frontend/API relationship.

61. OPTIONS / Preflight Requests

The backend supports:

```
OPTIONS
```

and responds to CORS preflight requests.

An unapproved origin receives:

```
403
```

instead of being granted credentialed access.

The auth common layer performs this check.

62. Rate Limiting

There is an application-level rate limiter:

```
api/security/rate-limit.php
```

The current rules are:

```
General API:  
90 requests / 60 seconds / IP  
  
Authentication endpoints:  
6 requests / 600 seconds / IP
```

The painting list endpoint is deliberately excluded from rate limiting.

CORS preflight requests are excluded.

Already authenticated and 2FA-verified administrators are excluded.

63. Rate Limiter Storage

The limiter uses files in the operating system's temporary directory rather than storing counters in the repository.

The basic concept is:

```
system temp directory
  ↓
aramane_rate_limits
  ↓
hashed IP bucket
  ↓
JSON state file
```

The file is locked with:

```
flock()
```

to protect concurrent access.

64. Rate Limiting Failure Behavior

If the server cannot create/open the temporary rate-limit file, the implementation intentionally fails open.

In other words:

```
rate limiter storage unavailable
  ↓
request continues
```

rather than:

```
rate limiter storage unavailable
  ↓
entire site unavailable
```

This is explicitly documented in the source.

65. Where the Rate Limiter Is Applied

`database.php` loads the rate limiter before opening the database connection.

Therefore the normal API processing path is broadly:

```
API endpoint
  ↓
database.php
  ↓
rate limiter
  ↓
PDO connection
```

This is a notable architectural choice because the limiter executes before database access is established.

66. Authentication Security Model

Customer:

```
PHP session cookie
HttpOnly
Secure when HTTPS
SameSite configurable
CORS origin restrictions
```

Admin:

```
Separate PHP session
HttpOnly
Secure when HTTPS
SameSite configurable
Admin type requirement
2FA verification requirement
```

Backend endpoints independently call:

```
requireCustomer()
```

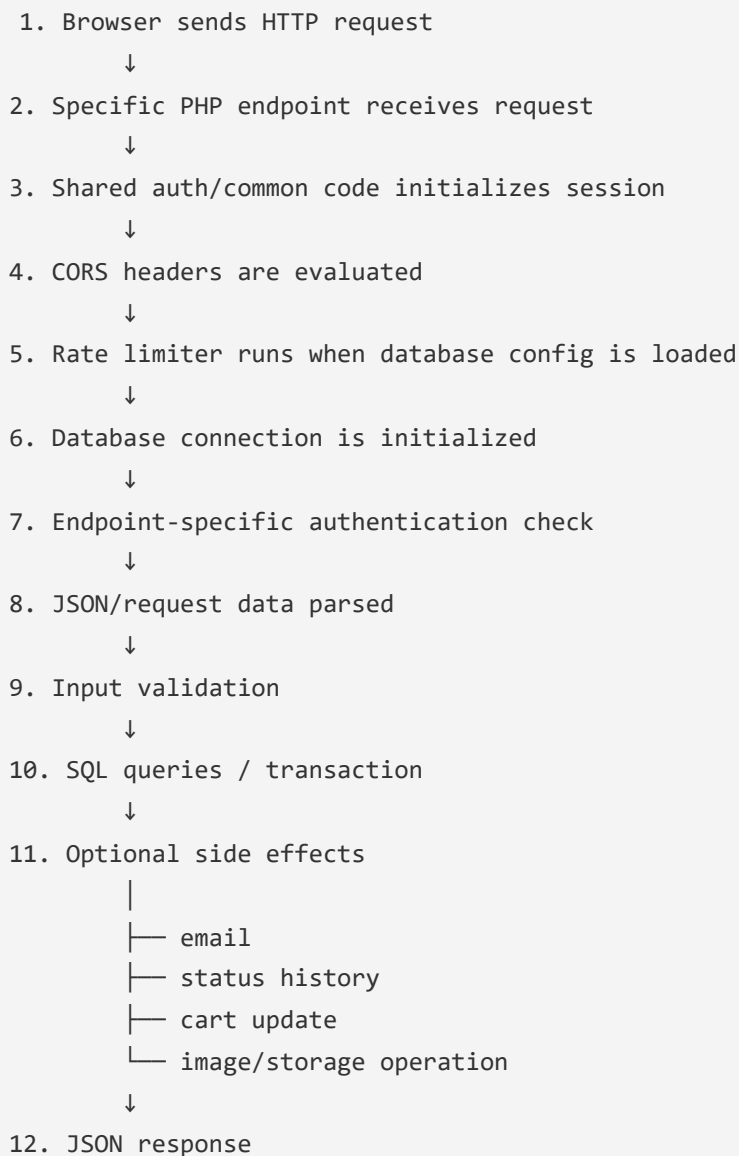
or:

```
requireAdmin()
```

rather than trusting the frontend UI.

67. Request Processing Pipeline — Generic

Almost every API request can be mentally traced like this:





13. React updates its UI

68. Example Request — Loading a Painting

```
Browser
|
| GET /paintings/get.php?id=25
v
PHP endpoint
|
+--> common configuration
|
+--> rate limiting
|
+--> database connection
|
+--> query painting
|
+--> query associated information/images
|
v
JSON response
|
v
PaintingDetails.jsx
|
+--> setPainting()
|
+--> setImages()
|
+--> render
```

The frontend's actual detail-page load is implemented in `PaintingDetails.jsx`.

69. Example Request — Add to Cart

```
User
|
| click "Add to Cart"
v
PaintingDetails.jsx
|
| POST /cart/add-item.php
| credentials: include
v
PHP
|
+--> customer session
|
+--> validate request
|
+--> database/cart update
|
v
JSON success
|
v
Toast
|
v
/cart
```

70. Example Request — Submit Order

```
Cart.jsx
|
| POST /orders/create.php
v
auth/_common.php
|
+--> session
+--> CORS
|
v
database.php
|
+--> rate limiter
```



```

+--> PDO
|
v
orders/create.php
|
+--> requireCustomer()
+--> validate contact data
+--> begin transaction
+--> lock cart
+--> resolve prices
+--> create order
+--> create order items
+--> create history
+--> delete cart
+--> commit
|
+--> mail notification
|
v
201 JSON
|
v
Cart.jsx
|
+--> toast
+--> navigate /track-orders

```

71. Example Request — Admin Status Update

```

Admin OrderDetails / OrderStatusManager
|
| POST /orders/update-status.php
v
api/admin/_common.php
|
+--> admin session
+--> 2FA verified?
v
database.php

```

```
|
+--> rate limiter
+--> PDO
v
orders/update-status.php
|
+--> validate
+--> lock order
+--> update status
+--> update notes/timestamps
+--> write history
+--> commit
|
+--> send email
v
JSON response
|
v
Admin UI refresh
```

72. Customer UI and Backend Are Not the Same Source of Truth

An important architectural principle in this codebase is:

```
Frontend = user experience
Backend = authoritative business rules
```

For example, even if the React UI disables a button, the backend endpoint still performs its own validation.

The cancellation endpoint verifies:

```
customer owns order
status permits cancellation
```

The admin status endpoint verifies:

```
admin session
2FA
valid status
order exists
order is not customer-cancelled
```

This is the correct general architectural direction for security.

73. Public vs Protected API Concepts

The backend can broadly be divided into:

Public/read-oriented

```
paintings/list.php
paintings/get.php
categories/list.php
frames/list.php
sizes/list.php
faqs/list.php
system/maintenance.php
```

Customer-protected

```
auth/user-*
cart/*
orders/create.php
orders/cancel.php
orders/my-orders.php
orders/get.php
orders/history.php
...
```

Admin-protected

```
admin/*
paintings/create.php
paintings/update.php
paintings/upload-images.php
metadata/*
orders/update-status.php
system/maintenance-toggle.php
```

The exact endpoint authorization is determined inside the PHP source, but this is the conceptual separation reflected in the repository.

74. Admin Placeholders / Current Implementation Status

The admin navigation currently exposes:

```
Customers
Payments
Reports
Audit Log
```

but those routes currently render placeholder pages rather than complete management implementations.

The placeholder component states that the section will be connected to the marketplace API next.

Therefore:

```
Admin UI exists
≠
Feature is fully implemented
```

This distinction matters when assessing the current system.

75. Payment Architecture — Current State

The current order creation flow does **not** take payment.

Instead:

```
customer submits order
    ↓
PENDING_ACCEPTANCE
    ↓
admin/artist review
    ↓
ACCEPTED
    ↓
PAYMENT_DUE
    ↓
PAID
```

The status model already contains:

```
PAYMENT_DUE
PAID
```

and there is an order endpoint called:

```
release-price.php
```

but the initial customer checkout itself only creates the order request and does not charge the customer.

The customer cart UI also explicitly represents the initial operation as an order request with no payment at that stage.

76. Frontend State Management

The application does not appear to use Redux, Zustand, MobX, or another global state framework.

The dominant pattern is local React state:

```
useState()  
useEffect()
```

plus browser events for lightweight cross-component updates, for example:

```
window.dispatchEvent(new Event('ara-cart-updated'))
```

This is used around cart mutations to notify other UI elements that the cart changed.

77. Toast / Feedback System

Both customer and admin applications contain reusable toast components/providers.

Customer:

```
Toast.jsx  
ToastProvider.jsx
```

Admin:

```
Toast.jsx
```

These are used for:

```
success  
error  
operation feedback
```

instead of relying solely on browser alerts.

78. Reusable UI Components

Both frontend applications contain generic reusable pieces such as:

```
UniversalDropdown  
UniversalPrompt  
Toast
```

The customer app also has:

```
Layout  
ToastProvider
```

while the admin app has additional painting-specific components such as:

```
PaintingSizeOptions
```

This indicates the frontends are componentized rather than placing every UI function into one monolithic page.

79. SEO / Metadata Structure

The customer application contains:

```
public/aa-og-img.svg  
public/aa-ogimage.png  
robots.txt  
sitemap.xml
```

and the frontend dynamically updates metadata.

The current painting detail metadata path works like:

```
route /paintings/:id  
  ↓  
MetadataManager  
  ↓  
GET /paintings/get.php?id=<id>  
  ↓
```

```
painting name + description
  ↓
document title / description / OG tags
```

The admin application also has:

```
metadata-enhancer.js
Metadata.jsx
```

showing that SEO metadata is treated as a managed feature.

80. Deployment Topology

The source clearly indicates a production topology resembling:

```
Customer:
https://arts.araha.co.in

Admin:
https://artsadmin.araha.co.in

API:
https://api.arts.araha.co.in
```

The frontend API configuration hardcodes the production API host, while the auth common files explicitly include the customer/admin origins in the allowed-origin list.

81. Production Request Topology

A real customer request therefore looks conceptually like:

```
https://arts.araha.co.in
  |
  | fetch()
  |
```



```
      v
https://api.arts.araha.co.in
      |
      | PHP session cookie
      |
      v
MySQL
```

Admin:

```
https://artsadmin.araha.co.in
      |
      | fetch()
      |
      v
https://api.arts.araha.co.in
      |
      v
MySQL
```

82. Environment / Secrets Architecture

The repository contains:

```
customer/.env.example
admin/.env.example
api/.env.example
```

Production database credentials are intended to come from an untracked:

```
api/config/production.php
```

or environment variables.

The database connection code explicitly documents `production.php` as something that should not be committed.

That is the correct general separation:

```
source code
≠
production secrets
```

83. Important Current Code Observation — Customer Cart Image URLs

There is a notable inconsistency in the current customer frontend.

`PaintingDetails.jsx` resolves relative image URLs using the API origin:

```
new URL(apiUrl('')).origin + relative URL
```

whereas `Cart.jsx` currently resolves a relative image using:

```
http://localhost + relative URL
```

That means a relative image URL returned to the cart can resolve incorrectly in production.

Conceptually:

```
Painting Details:
relative image
  ↓
production API origin
  ↓
correct production URL

Cart:
relative image
  ↓
http://localhost...
  ↓
potentially broken in production
```

This is worth fixing because it is environment-specific behavior.

84. Important Current Code Observation — Duplicate Painting API Fetch

A painting detail route can involve two separate frontend consumers:

```
MetadataManager
  ↓
paintings/get.php

PaintingDetails.jsx
  ↓
paintings/get.php
```

So a single painting detail navigation can result in:

```
request 1 → metadata
request 2 → page content
```

The data is not currently centrally shared between these two consumers.

85. Important Current Code Observation — Maintenance Polling

The customer application checks maintenance status approximately every two seconds.

Conceptually:

```
Client 1 ┌
Client 2 ┌
Client 3 ┌
Client 4 ┌
           └─> maintenance.php
Client N ┌
```

This provides rapid state propagation but creates continuous request traffic.

The rate limiter currently does not exempt the maintenance endpoint, while the painting collection endpoint is explicitly exempt.

86. Important Current Code Observation — Order Quantity

The backend order calculation supports:

```
quantity
```

and calculates:

```
unit price × quantity
```

But the current customer cart interaction sends:

```
quantity: 1
```

and the visible cart UI is focused on choosing a size rather than changing quantity.

Therefore, the underlying data model supports quantity, while the current customer experience effectively treats a cart line as one artwork.

87. Important Current Code Observation — **REJECTED** Cancellation

The system supports an admin status:

REJECTED

and records it through the normal status-history mechanism.

However:

```
orders/cancel.php
```

does not classify `REJECTED` as a cancellation-blocking status.

Therefore:

```
ADMIN:  
PENDING_ACCEPTANCE → REJECTED
```

then

```
CUSTOMER:  
REJECTED → CANCELLED
```

is currently permitted by the cancellation endpoint.

This is a business-rule detail that should be explicitly decided rather than left implicit.

88. Request / Response Convention

The API generally uses JSON responses shaped around:

```
{  
  "success": true,  
  ...  
}
```

or:

```
{  
  "success": false,
```

```
"message": "..."  
}
```

The shared auth common layer defines:

```
jsonResponse(...)
```

and the admin common layer defines:

```
adminJsonResponse(...)
```

HTTP status codes are also used meaningfully:

```
201 Created  
400 Bad Request  
401 Unauthorized  
403 Forbidden  
409 Conflict  
422 Unprocessable Entity  
429 Too Many Requests  
500 Server Error
```

The exact endpoint determines which code is used.

89. Error Handling

Typical PHP endpoint behavior is:

```
validate request  
  ↓  
if invalid  
  ↓  
JSON error + appropriate HTTP status  
  
try  
  DB work  
catch Throwable  
  rollback
```

```
log internal error
return generic JSON error
```

For example, order creation explicitly rolls back an active transaction and logs the internal error before sending a generic 500 response.

This prevents internal SQL/error messages from being directly returned as the normal customer-facing response.

90. Transaction Model

Transactions are especially important in order operations.

Order creation:

```
BEGIN
↓
read/lock cart
↓
validate
↓
insert order
↓
insert items
↓
insert history
↓
clear cart
↓
COMMIT
```

Admin order status:

```
BEGIN
↓
lock order
↓
update order
↓
insert history
```

```
↓  
COMMIT
```

Customer cancellation:

```
BEGIN  
↓  
lock order  
↓  
update order  
↓  
insert history  
↓  
COMMIT
```

This makes order state changes substantially safer than performing a sequence of independent SQL queries without a transaction.

91. Request Flow by Feature

Browse paintings

```
Customer  
→ /paintings  
→ Paintings.jsx  
→ GET paintings/list.php  
→ MySQL  
→ JSON  
→ render cards
```

View painting

```
Customer  
→ /paintings/:id  
→ PaintingDetails.jsx  
→ GET paintings/get.php
```


- MySQL
- JSON
- gallery/details

Login

- Customer
- Login.jsx
 - POST auth/user-login.php
 - authenticate
 - create customer session
 - JSON
 - /account

Register

- Customer
- Login.jsx registration mode
 - POST auth/user-register.php
 - create user
 - session/auth result
 - /account

Password reset

- Customer
- request reset
 - POST auth/request-password-reset.php
 - mail/reset token workflow
 - email
 - reset-password route

Add item

- PaintingDetails
- POST cart/add-item.php
 - session check

- DB
- success

Change size

- Cart
- POST cart/update-item.php
- DB
- reload cart

Remove item

- Cart
- POST cart/remove-item.php
- DB
- reload cart

Create order

- Cart
- POST orders/create.php
- transaction
- order + items + history
- clear cart
- notification
- /track-orders

Customer cancel

- OrderDetails
- POST orders/cancel.php
- verify ownership
- status validation
- CANCELLED
- history
- notification

Admin login

```
AdminLogin
→ admin/authentication endpoints
→ admin session
→ OTP / 2FA state
→ admin session verification
```

The admin backend exposes OTP and password-reset operations under both `api/admin` and authentication-related endpoints.

Admin painting creation

```
AddPainting
→ painting creation endpoint
→ DB
→ image uploads
→ sizes / metadata
→ refresh
```

Admin order update

```
Orders
→ OrderDetails
→ OrderStatusManager
→ POST orders/update-status.php
→ transaction
→ status history
→ mail
→ UI refresh
```

92. Mental Model of the Whole System

The easiest way to understand this project is to divide it into five layers:

```
LAYER 1
React pages
  ↓
LAYER 2
fetch() API calls
  ↓
LAYER 3
PHP endpoint
  ↓
LAYER 4
shared auth / rate limiting / PDO
  ↓
LAYER 5
MySQL + external side effects
```

For an order:

```
Cart.jsx
  ↓
orders/create.php
  ↓
requireCustomer()
  ↓
rate-limit.php
  ↓
database.php
  ↓
PDO/MySQL
  ↓
orders + order_items + history
  ↓
mail/
```

For an admin status change:

```
OrderStatusManager.jsx
  ↓
orders/update-status.php
  ↓
requireAdmin()
  ↓
rate-limit.php
  ↓
database.php
```

↓
MySQL transaction
↓
order history
↓
mail/

93. Most Important Files to Understand First

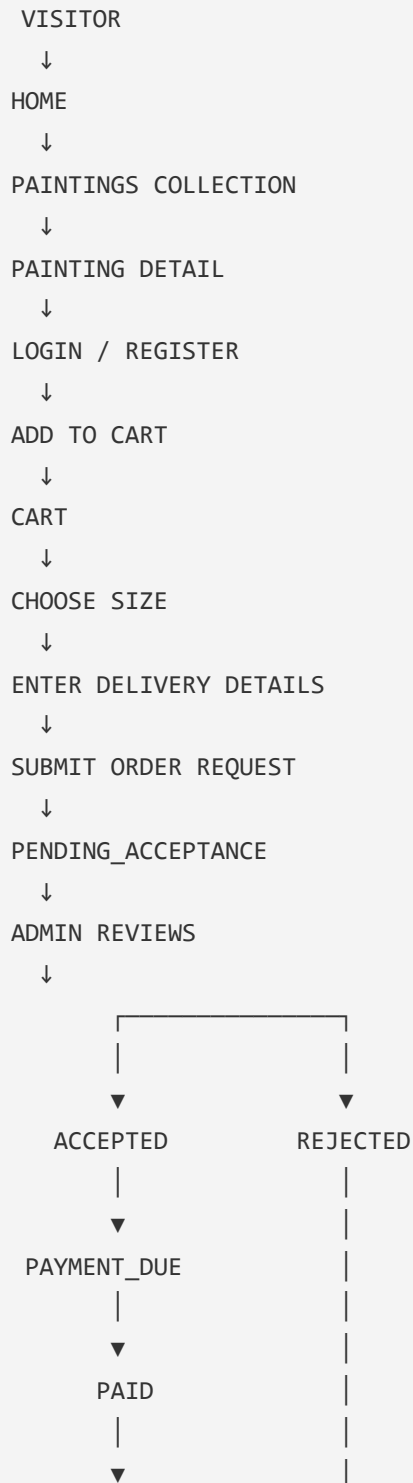
If a developer needs to understand this project quickly, the most useful reading order is:

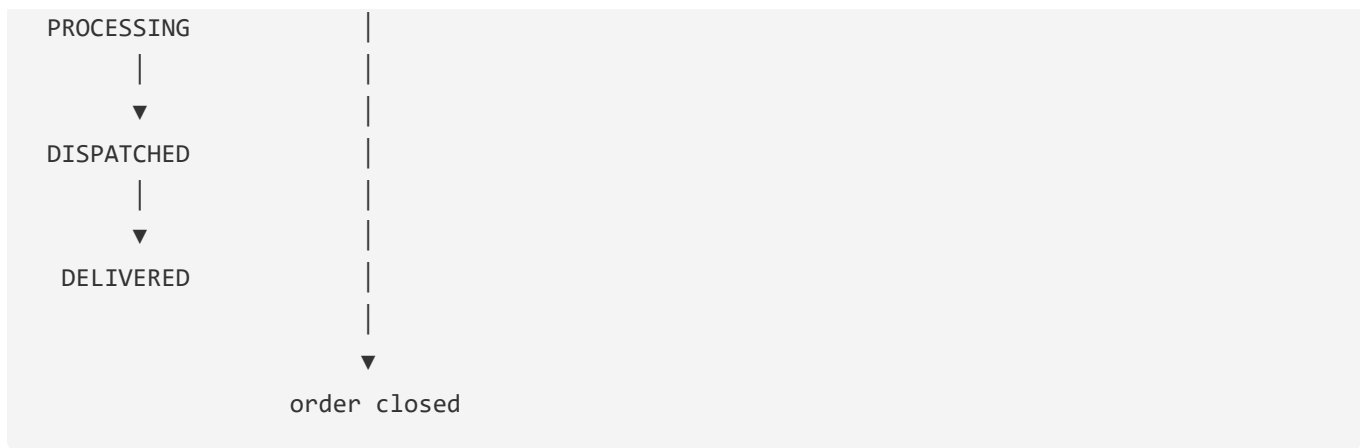
1. customer/src/App.jsx
2. customer/src/config/api.js
3. customer/src/pages/PaintingDetails.jsx
4. customer/src/pages/Cart.jsx
5. admin/src/App.jsx
6. api/auth/_common.php
7. api/admin/_common.php
8. api/config/database.php
9. api/orders/create.php
10. api/orders/update-status.php
11. api/orders/cancel.php
12. api/security/rate-limit.php
13. api/mail/index.php
14. api/mail/Mailer.php
15. api/mail/Templates.php
16. api/migrations/*

Those files reveal most of the application's architecture.

94. Overall System Flow

The complete customer journey is:

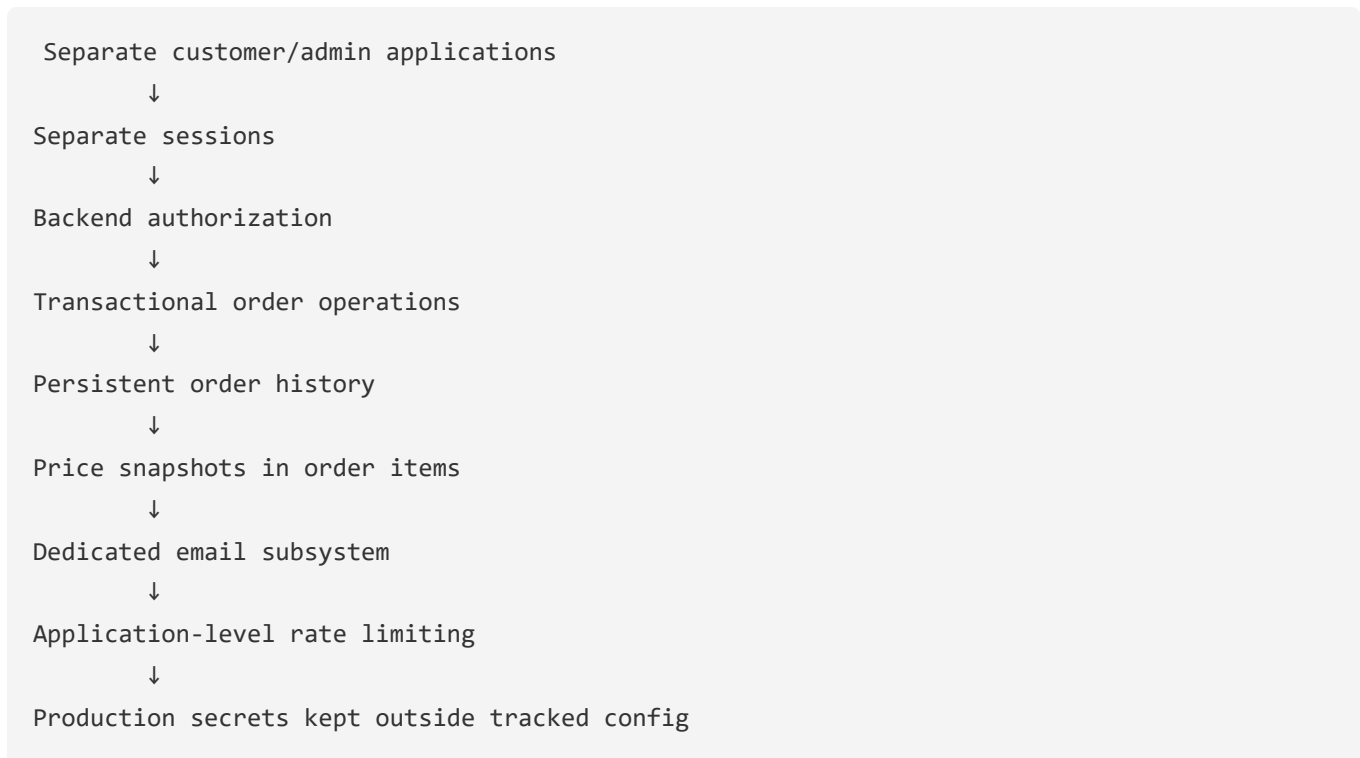




The current backend, however, does not enforce every conceptual transition as a strict state machine. The status endpoint accepts the defined status values, with selected protections around cancelled orders.

95. Architectural Strengths

The current architecture has several useful properties:



These make the application easier to extend while keeping important business rules on the server.

96. Architectural Areas to Watch

The main areas worth reviewing as development continues are:

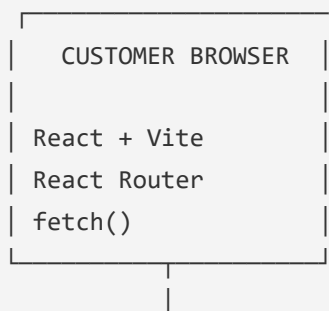
1. Formalizing order state transitions
2. Ensuring rejected orders obey the desired cancellation rule
3. Removing production localhost URL fallbacks from customer image handling
4. Completing currently-placeholder admin sections
5. Avoiding duplicate API requests where metadata and page data overlap
6. Evaluating the frequency of maintenance polling
7. Ensuring all sensitive production configuration stays outside Git
8. Keeping frontend and backend validation rules synchronized

97. One-Sentence Description of the System

The project is best described as:

A React/Vite customer marketplace and separate React/Vite administration console communicating with a session-based PHP/MySQL API that manages paintings, size options, carts, orders, authentication, metadata, maintenance mode, status history and transactional email notifications.

98. Final Architecture Diagram



https://arts.araha.co.in

API calls

PHP API SERVER

https://api.arts.araha.co.in

CORS / Sessions

Rate Limiting

Domain Endpoints

auth/

admin/

paintings/

cart/

orders/

categories/

sizes/

frames/

metadata/

faqs/

system/

PDO / MySQL

USERS

carts

sessions

ORDERS

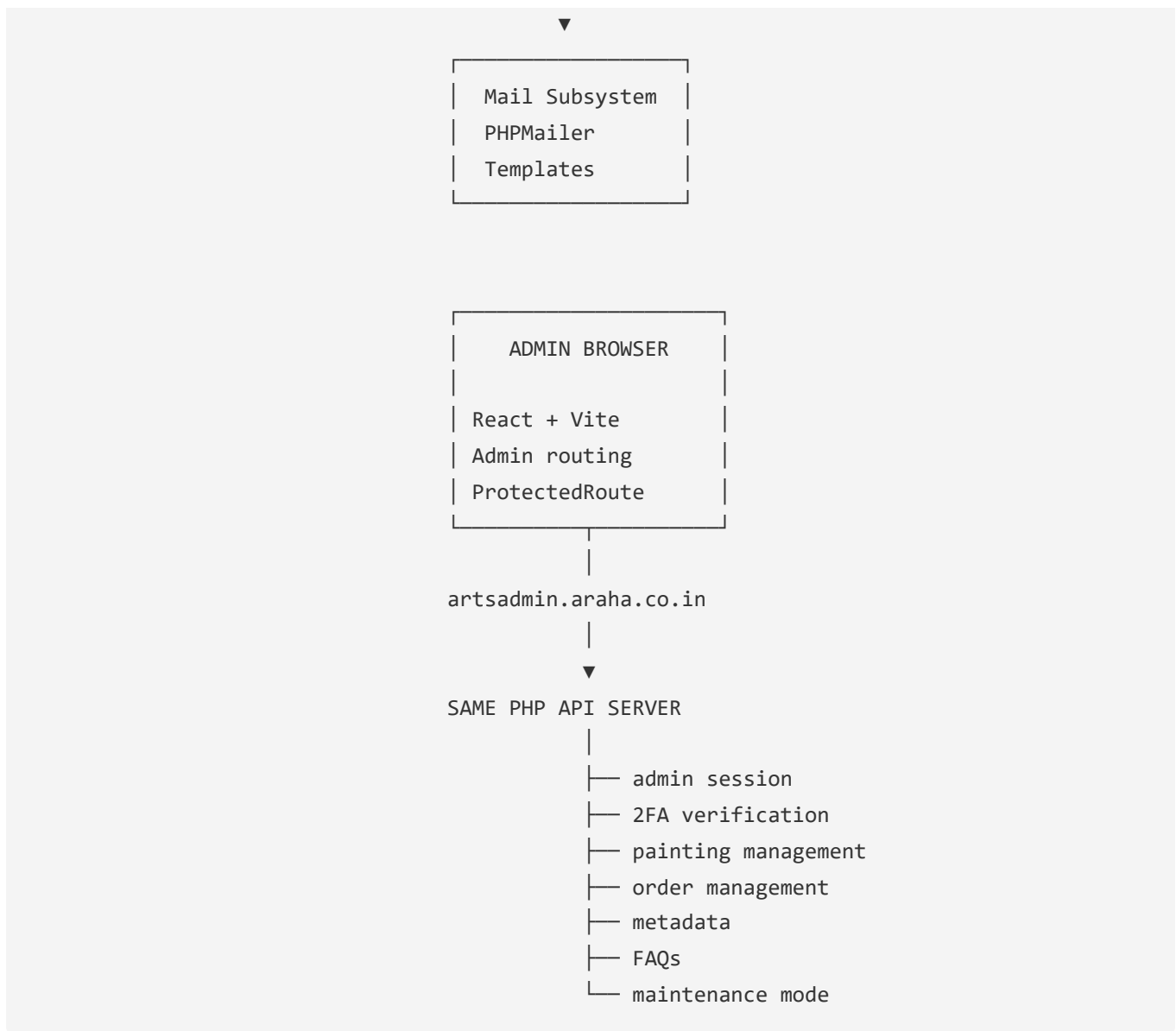
items

history

PAINTINGS

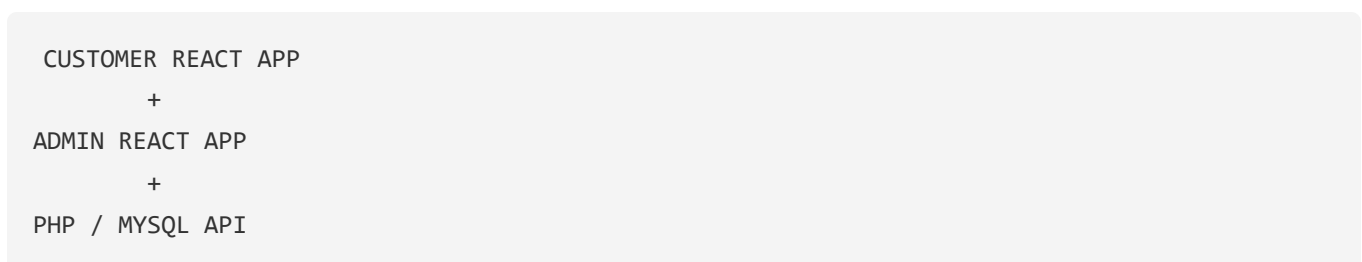
images

sizes



99. Bottom Line

The repository is built around a straightforward three-application architecture:



The API is the central authority.

The most important domain is the order lifecycle:

Cart

- Order Request
- Pending Acceptance
- Admin/Artist Decision
- Payment Due
- Paid
- Processing
- Dispatched
- Delivered

with:

```
order_items
order_status_history
transactional updates
email notifications
```

forming the backbone of the commerce logic.

The authentication model is session-based, with separate customer and admin sessions; admin operations additionally require verified 2FA state.

The repository also already contains the foundations for:

```
multi-size paintings
image galleries
SEO metadata
Google login
password reset
maintenance mode
email notification groups
rate limiting
order history
administrator management
```

The principal implementation issue visible from this source review is that some behavior still needs to be formalized at the business-rule level, especially the exact order state transition rules and rejected-order cancellation behavior.